

计算机系统基础 作业一

学号：2410668

姓名：吕昊远

一、浮点数有没有移位操作和扩展操作？为什么？

浮点数没有直接的移位操作，但是存在类型扩展操作。

原因：浮点数与整型结构不同，由符号位、指数位、尾数位三部分组成，直接对整个浮点数的二进制位进行移位会破坏其结构，并不能实现乘二或者除二的效果；扩展操作上，浮点数存在由 float 转 double 的类型扩展，但是具体操作与整数不同，并非直接填充位，而是根据 IEEE 754 标准重新编码。指数位从 8 位扩展到 11 位，保持指数数值不变；尾数位则是从 23 位扩展到 52 位，低位补 0 来保证原尾数的有效精度。

二、为什么整数除 0 会发生异常而浮点数除 0 不会？

整数除 0 时，会触发 CPU 指令集中的除法，而指令集中并没有定义整数除以 0 的行为，最终会被操作系统捕获为异常，然后终止程序。

浮点数除 0 时，IEEE 754 标准已经预定义了特殊值来处理这种情况，结果会以特殊值的方式表示，如正负无穷和 $0 \div 0$ 输出的 NaN，这些都是合法的浮点数表示，硬件不会触发异常，而是直接返回对应特殊值。

三、为什么同一个实数赋值给 float 型变量和 double 型变量，输出结果会有所不同呢？

主要是因为两者的存储精度不同，大多数的实数都没办法用有限的二进制数表示，只能存储一个近似值，而 double 比 float 精度更高误差更小，因而所得近似值也不一样，两者多数情况下的数值都是存在一定差异的。

四、为什么 float 情况下输出的结果会比原来的大？这到底有没有根本性原因还是随机发生的？为什么会出现这样的情况？

不是随机的，其根本性原因在于浮点数的舍入规则。对于一个无限循环的二进制小数，float 由于精度有限必须对小数进行舍入处理。而 IEEE 754 默认采用就近舍入到偶数的方式，即舍入到最近的可以表示的数，如果正中间就舍入到偶数。当原数的二进制近似值与两个可表示值的距离不相等时，且更接近的可表示值比原数大，float 存储的结果就会比原数大。这种偏差是由原数的二进制形式和舍入规则共同决定的，可以通过计算预测出偏大还是偏小，并非随机发生。